# ICG Reportería Dinámica

Documento técnico, arquitectónico y operativo

| | |
|---|---|
| **Repositorio** | github.com/arenas037/ICG_Reporteria (privado) |
| **Versión** | Base funcional completa — apta para piloto en cliente |
| **Fecha** | 4 de agosto de 2026 |
| **Plataforma** | Node.js + React sobre SQL Server (Windows Server) |
| **Licenciamiento** | Ningún componente de pago |
| **Destinatario** | Responsable técnico entrante |

*Documento de uso interno. Describe la estructura de datos de un ERP de terceros y el diseño de los controles de seguridad de la herramienta; no debe publicarse.*

# 1. Resumen ejecutivo

ICG Reportería Dinámica es una herramienta web que permite construir, ejecutar, programar y distribuir reportes y tableros sobre las bases de datos del ERP ICG, sin instalar nada en el equipo del usuario final y sin incurrir en costos de licenciamiento.

Sustituye el patrón habitual de "cada consulta nueva es un desarrollo": un usuario con perfil de creador define un reporte una sola vez — consulta, parámetros, formatos, agrupaciones — y el resto de la organización lo consume, lo exporta o lo recibe por correo de forma programada.

## Las tres ideas que definen el sistema

1. **No se toca la base de datos del ERP.** Toda la configuración vive en una base independiente, ICG_Reporteria_Config. No se crea ni una tabla dentro de las bases de ICG. Una actualización del ERP no encuentra objetos ajenos, y desinstalar la herramienta es eliminar una base de datos.
2. **Es de solo lectura, y está garantizado por tres muros independientes.** No basta con que la aplicación "no escriba": el login SQL que usa carece de permisos de escritura, de modo que aunque alguien lograra colar una sentencia destructiva, el motor de base de datos la rechazaría.
3. **El destino de cada empresa se resuelve en vivo.** La herramienta lee de ICG a qué servidor y base pertenece cada empresa en el momento de ejecutar. No duplica cadenas de conexión, así que mover una empresa de servidor no requiere reconfigurar nada.

## Tamaño y estado

| Área | Archivos | Líneas | Contenido |
|---|---|---|---|
| Backend (Node.js) | 39 | 4.540 | API, motor de consultas, servicios, scheduler |
| Frontend (React) | 34 | 5.213 | Interfaz completa, 20 pantallas |
| Base de datos | 14 | 1.590 | Esquema, catálogo semántico, migraciones |
| Pruebas | 17 | 1.944 | 163 pruebas automáticas + 6 guiones manuales |
| **Total** | **106** | **13.606** | |

> **Estado actual**
>
> La base funcional está completa y probada contra una instalación real de ICG sobre SQL Server 2022. Las 163 pruebas automáticas pasan. Lo que queda pendiente son ampliaciones de alcance —más orígenes de datos en el catálogo, manipulación libre de objetos en el tablero—, no correcciones para poder operar. El detalle está en el capítulo 14.

# 2. Alcance

Este capítulo delimita qué resuelve la herramienta y qué queda deliberadamente fuera. Los límites del segundo bloque no son carencias pendientes de implementar: son decisiones de diseño que sostienen la seguridad del conjunto, y modificarlas exige revisar la arquitectura entera.

## 2.1 Dentro del alcance

| Capacidad | Detalle |
|---|---|
| Reportes simples | Una consulta, parámetros dinámicos, rejilla con formatos, agrupación y subtotales. |
| Reportes multinivel | Cabecera y detalle enlazados (maestro-detalle) en una sola definición. |
| Tableros (dashboards) | Objetos visuales sobre un catálogo de datos, sin escribir SQL, con filtrado cruzado. |
| Parámetros | Texto, número, fecha, lista fija y lista alimentada por consulta; selección múltiple. |
| Multiempresa | Gestión y contable, con ejercicio, respetando los permisos que el usuario tiene en ICG. |
| Exportación | PDF, Excel, CSV, HTML, JSON y XML. El PDF admite orientación y tamaño de hoja. |
| Envío por correo | Puntual desde el visor, o programado por expresión cron. |
| Programación | Scheduler propio con cola en SQL Server, tolerante a caídas y a varios servidores. |
| Identidad corporativa | Logo, encabezado y pie por empresa, incrustados en las exportaciones. |
| Auditoría | Quién hizo qué, cuándo y desde qué IP, incluidos los intentos rechazados. |
| Versionado | Historial completo de cada reporte, con comparación entre dos versiones. |
| Usuarios y roles | Cuatro permisos independientes, sobre los usuarios que ya existen en ICG. |
| Perfiles | Agrupan reportes y se asignan a usuarios; determinan qué ve cada quien. |
| Portabilidad | Exportar la definición de un reporte e importarla en la instalación de otro cliente. |
| Editor T-SQL | Monaco integrado, con realce de parámetros y detección automática de columnas. |

## 2.2 Fuera del alcance, por diseño

| Queda fuera | Razón |
|---|---|
| **Escribir, borrar o truncar datos** | La herramienta es de solo lectura. Un proceso que deba modificar datos se implementa como procedimiento almacenado fuera de la herramienta, revisado por un DBA. |
| **Crear usuarios, bases de datos o procedimientos** | Son operaciones de administración del ERP. La interfaz no las expone ni permite expresarlas en SQL. |
| **Ejecutar cualquier procedimiento almacenado** | Un reporte sí puede ejecutar un SP, pero solo si está registrado en la lista blanca. No existe EXEC libre. |
| **Administrar usuarios de ICG** | Los usuarios viven en ICG y se leen sin modificarlos. Aquí solo se decide qué puede hacer cada uno dentro de la reportería. |

| Queda fuera | Razón |
|---|---|
| **Otros motores de base de datos** | El sistema es exclusivo de SQL Server: usa T-SQL, tipos y bloqueos propios del motor. |

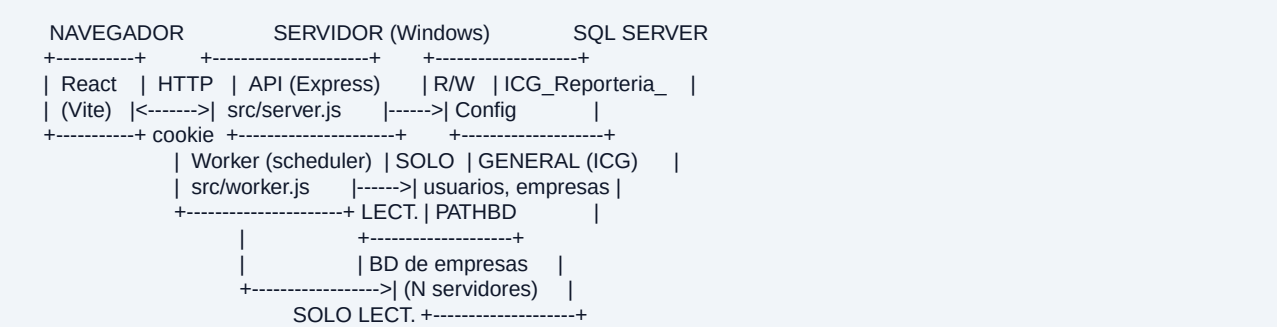
**Regla que no debe relajarse**

Cuando aparezca un requerimiento que exija escribir en la base —recalcular un saldo, marcar registros procesados—, la respuesta correcta es un procedimiento almacenado creado y revisado fuera de esta herramienta, invocado desde la lista blanca si hiciera falta. Ampliar los permisos del login SQL o abrir la guarda de SQL desmonta los tres muros de seguridad a la vez.

# 3. Arquitectura

## 3.1 Vista general

Tres piezas ejecutables y tres tipos de base de datos:




- **API** — atiende al navegador. Autentica, autoriza, valida el SQL, ejecuta y exporta.
- **Worker** — proceso aparte que ejecuta lo programado. Puede haber varios en paralelo.
- **Frontend** — se compila a archivos estáticos; en producción los sirve el mismo servidor.

> **Por qué el worker es un proceso separado**
>
> Un reporte programado sobre un año de ventas puede tardar minutos. Si se ejecutara dentro del API, ocuparía el mismo proceso que atiende a los usuarios interactivos y la aplicación se sentiría lenta. Separarlos permite además reiniciar el API sin perder trabajos en cola.

## 3.2 El enrutamiento por PATHBD

Es la decisión central del diseño y conviene entenderla antes que nada.

En ICG, la base GENERAL guarda el destino de cada empresa en el campo PATHBD, con el formato SERVIDOR:BASEDATOS. La herramienta lo lee en el momento de ejecutar y crea la conexión correspondiente:

```
GENERAL.EMPRESAS
 CODEMPRESA  TITULO          PATHBD
 1           Tienda Centro   SRVSQL01:CLARKS2025
 2           Tienda Este     SRVSQL02:CLARKS2025_E

 --> al ejecutar un reporte de la empresa 2, la conexión se abre
  contra SRVSQL02, base CLARKS2025_E, con credencial de solo lectura.
```

> **Consecuencia operativa**
>
> Si el cliente mueve una empresa a otro servidor, basta con que ICG actualice su PATHBD: la reportería lo toma sin tocar configuración. No existe ninguna cadena de conexión duplicada que pueda quedar desfasada. Lo único que se guarda por servidor es la credencial de lectura, cifrada, en rep.ServidorCredencial.

## 3.3 Gestión de conexiones

connectionManager.js administra tres clases de conexión con ciclos de vida distintos:

| Conexión | Destino | Permisos | Ciclo de vida |
|---|---|---|---|
| configPool | ICG_Reporteria_Config | Lectura y escritura | Fija, se crea una vez |
| generalPool | GENERAL de ICG | Solo lectura | Fija, se crea una vez |
| dataPools | BD de cada empresa | Solo lectura | Al vuelo, caché con vencimiento |

Los pools de datos se crean bajo demanda y se guardan en caché por clave servidor::base. Un barrido periódico cierra los que llevan más de 30 minutos sin uso (configurable), para que una instalación con decenas de empresas no acumule conexiones ociosas. Además se abren con readOnlyIntent, lo que permite aprovechar réplicas de solo lectura si el cliente las tiene.

## 3.4 Aislamiento de la configuración

La base ICG_Reporteria_Config contiene 33 tablas y 5 vistas en el esquema rep. Agrupadas por función:

| Grupo | Tablas |
|---|---|
| Reportes | Reporte, ReporteConsulta, ReporteParametro, ReporteParametroOpcion, ReporteColumna, Carpeta |
| Catálogo semántico | CatalogoOrigen, CatalogoCampo, CatalogoRelacion, CatalogoVersion |
| Tableros | Dashboard, DashboardConfig, DashboardCompartido |
| Seguridad | AppRol, UsuarioApp, UsuarioRolHistorial, Perfil, PerfilReporte, UsuarioPerfil, ProcedimientoPermitido |
| Auditoría | AuditoriaEvento, ReporteVersion, EjecucionLog, PortabilidadLog |
| Programación | Programacion, JobCola, EnvioCorreoHistorial |
| Infraestructura | ServidorCredencial, ConfiguracionApp, ConfiguracionCorreo, Empresa, EmpresaBranding, UsuarioPreferencia |

# 4. Seguridad

Es el capítulo más importante del documento. La herramienta permite que un usuario escriba T-SQL y lo ejecute contra la base de producción del ERP: sin controles sería un riesgo inaceptable. El diseño parte de asumir que cualquiera de las defensas puede fallar, y por eso son independientes entre sí.

## 4.1 Los tres muros

| # | Muro | Qué detiene | Dónde vive |
|---|---|---|---|
| 1 | Login SQL de solo lectura | Cualquier escritura, aunque burle todo lo demás | SQL Server |
| 2 | sqlGuard | SQL peligroso antes de enviarlo al motor | backend/src/engine/sqlGuard.js |
| 3 | Parámetros enlazados | Inyección a través de los valores | queryTemplateEngine.js |

> **El orden importa**
>
> El primer muro es el único que no depende de código propio. Si mañana apareciera un error en sqlGuard, el login sin permisos de escritura seguiría impidiendo el daño. Por eso el despliegue con un login administrador —"total, la aplicación no escribe"— es la peor decisión posible: elimina la única garantía que no depende de nosotros.

### Muro 1 — Privilegio mínimo en SQL Server

El despliegue crea dos logins separados, que nunca deben acumular más permisos:

| Login | Permisos | Alcance |
|---|---|---|
| readonly_rep | db_datareader | GENERAL y bases de empresas de ICG |
| icg_reporteria_app | Lectura y escritura | Únicamente ICG_Reporteria_Config |

El login de la aplicación no tiene ningún acceso a las bases de ICG, y el de lectura no puede escribir en ninguna parte. Ni siquiera un fallo total de la aplicación permitiría modificar datos del ERP.

### Muro 2 — La guarda de SQL

Todo T-SQL escrito por un usuario pasa por sqlGuard, tanto al guardarlo como al ejecutarlo. Validar en ambos momentos evita que una definición aprobada bajo reglas antiguas siga ejecutándose tras endurecerlas. El procedimiento tiene cuatro pasos:

1. **Normalizar.** Se eliminan comentarios y literales de cadena. Así DR/**/OP no se disfraza de texto inocuo, y a la vez la palabra 'DROP' dentro de una cadena legítima no genera un falso positivo. Los comentarios de bloque se cuentan anidados, como hace T-SQL.
2. **Una sola sentencia.** El punto y coma solo se tolera al final. Impide encadenar una consulta inocente con otra destructiva.
3. **Lista blanca de inicio.** La consulta debe empezar por SELECT o WITH. Todo lo demás se rechaza antes de mirar nada más.
4. **Lista negra de operaciones.** Sobre el texto ya normalizado se buscan palabras completas: INSERT, UPDATE, DELETE, MERGE, TRUNCATE, DROP, CREATE, ALTER, GRANT, REVOKE, BACKUP, RESTORE, DBCC, SHUTDOWN, KILL, WAITFOR, OPENROWSET, los prefijos sp_ y xp_, y SELECT ... INTO, que crearía una tabla.

Cada rechazo se registra en la auditoría como SQL_RECHAZADO, con el texto y el motivo. Ese registro es a la vez rastro de seguridad y ayuda al soporte.

> **Por qué también se prohíben COMMIT, ROLLBACK y WAITFOR**
>
> No aportan nada a un reporte y sí sirven para ocultar efectos o bloquear el servidor: WAITFOR DELAY mantiene abierta una conexión indefinidamente y es un vector de denegación de servicio trivial.

### Muro 3 — Los valores nunca se concatenan

Los parámetros se escriben en la consulta con llaves y el motor los convierte en parámetros enlazados de SQL Server, con su tipo declarado:

```
El creador escribe:    WHERE FECHA BETWEEN {FECHA_INI} AND {FECHA_FIN}
                       AND CODALMACEN IN ({ALMACENES})

Lo que se ejecuta:     WHERE FECHA BETWEEN @FECHA_INI AND @FECHA_FIN
                       AND CODALMACEN IN (SELECT value FROM STRING_SPLIT(@ALMACENES, ','))

El valor viaja aparte, tipado. No hay concatenación en ningún punto.
```

La selección múltiple se resuelve con STRING_SPLIT en lugar de construir la lista en texto. Cuando un reporte necesita un identificador dinámico — ordenar por una columna elegida por el usuario —, ese identificador se valida contra una lista blanca de columnas conocidas y se envuelve en QUOTENAME: nunca se interpola texto libre.

## 4.2 Reportes basados en procedimiento almacenado

Un reporte puede ejecutar un procedimiento almacenado, pero solo si está registrado en rep.ProcedimientoPermitido. La sintaxis admitida es deliberadamente estrecha:

```
EXEC dbo.REP_VENTASDIARIAS @Desde = {FECHA_INI}, @Hasta = {FECHA_FIN}
```

Se rechaza todo lo demás: procedimientos no autorizados, los prefijos sp_ y xp_ aunque alguien los registre, EXEC con SQL dinámico entre paréntesis, y cualquier encadenamiento. Los valores siguen viajando enlazados. La pantalla de administración permite descubrir los procedimientos de la base activa — leyendo solo el catálogo del sistema— y autorizarlos con un clic; cada alta y cada baja quedan auditadas.

> **Límite real de la lista blanca**
>
> Un procedimiento almacenado hace internamente lo que quiera. La lista blanca impide que se llame cualquier cosa, pero la garantía de que no escriba proviene del login de solo lectura. Autorice únicamente procedimientos de lectura revisados por un DBA, y preste especial atención a los que usen EXECUTE AS, porque cambian la identidad efectiva y podrían eludir el primer muro.

## 4.3 Autenticación

El usuario se identifica solo con su contraseña, sin escribir nombre de usuario. Es el patrón que ICG usa en su propio cliente, y se verificó contra la instalación real antes de adoptarlo: 309 usuarios activos con clave, 309 claves distintas, ninguna colisión.

Las contraseñas se validan contra GENERAL.USUARIOS.NEWPASS, replicando el cifrado propio de ICG (una clave fija de nueve bytes con desplazamiento por posición). Se excluyen los usuarios descatalogados, bloqueados y sin contraseña; este último filtro es necesario, porque de lo contrario una contraseña vacía

coincidiría con muchos usuarios a la vez.

Si alguna vez dos usuarios llegaran a compartir contraseña, el sistema rechaza el acceso en lugar de adivinar de quién se trata. Los intentos fallidos se auditan sin registrar jamás la contraseña probada.

Tras autenticarse, el usuario elige entre las empresas de gestión que ICG le autoriza. Esa elección se revalida en el servidor: manipular la petición desde el navegador no da acceso a una empresa ajena.

## 4.4 Autorización

Dos dimensiones independientes, que conviene no confundir:

| Dimensión | Responde a | Se configura en |
|---|---|---|
| Rol | ¿Qué puede hacer? | Pantalla Usuarios — 4 permisos independientes |
| Perfil | ¿Qué reportes ve? | Pantalla Perfiles — agrupa reportes y se asigna a usuarios |

Los permisos son PuedeCrear, PuedeEditar, PuedeProgramar y PuedeAdministrar. Este último es independiente de los demás a propósito: un creador de reportes no puede promoverse a sí mismo ni alterar los roles de otros.

Reglas de visibilidad: un usuario sin perfiles asignados ve todos los reportes — para no romper instalaciones existentes al introducir la función—; con perfiles, solo los de sus perfiles; los administradores ven todo siempre.

### Salvaguardas probadas

- Nadie puede cambiar, quitar ni desactivar su propio rol.
- No se puede degradar, desactivar ni eliminar al último administrador que quede.
- La administración del catálogo de datos está restringida al usuario SUPERVISOR (código 0), configurable por variable de entorno.

> **Recuperación de acceso**
>
> La variable ADMIN_LOGINS del archivo .env lista los usuarios de ICG que son administradores siempre, aunque no estén registrados en la herramienta. Resuelve el primer arranque y permite recuperar el control si una instalación se quedara sin administradores. Cambiarla exige acceso al servidor, no a la aplicación: por eso es un mecanismo de recuperación y no un agujero.

## 4.5 Secretos

Las contraseñas de conexión por servidor y las credenciales de correo se guardan cifradas con AES-256-GCM, con la llave en el archivo .env. El .env nunca se versiona: el repositorio solo contiene .env.example con los valores en blanco.

> **Al cambiar la llave de cifrado**
>
> APP_CRYPTO_KEY descifra lo que ya está guardado en la base. Si se rota, hay que volver a introducir las contraseñas de rep.ServidorCredencial y las de correo, porque los valores antiguos dejan de poder descifrarse. El script install/rotar-credenciales.sql cubre la rotación de los logins SQL, que es independiente de esto.

# 5. El motor de reportes

## 5.1 Tipos de reporte

| Tipo | Estructura | Uso típico |
|---|---|---|
| Simple | Una consulta, una rejilla | Listados, existencias, ventas por período |
| Multinivel | Consulta de cabecera + consulta de detalle | Facturas con sus líneas, pedidos |
| Tablero | Objetos visuales sobre el catálogo | Seguimiento gerencial, análisis exploratorio |

## 5.2 Ciclo de ejecución

1. Se resuelve la empresa activa y se revalida el permiso del usuario contra ICG.
2. Se lee PATHBD y se obtiene el pool de conexión correspondiente.
3. Se recupera la definición y se vuelve a validar el SQL con sqlGuard.
4. Se sustituyen los parámetros por marcadores enlazados con su tipo declarado.
5. Se ejecuta con tiempo límite (120 segundos por defecto).
6. Se aplican los formatos y subtotales configurados en cada columna.
7. Se registra la ejecución: quién, qué reporte, qué empresa, duración y filas.

## 5.3 Parámetros

Cada parámetro declara tipo, si es obligatorio, valor por defecto y cómo se muestra. Las listas pueden ser fijas o alimentarse de una consulta —que pasa por la misma guarda de seguridad, sin excepción—. La interfaz genera el control adecuado a cada tipo.

## 5.4 Editor de columnas

El botón "Detectar columnas" ejecuta la consulta en modo diseño y lee los metadatos del resultado, de modo que el creador no escribe los nombres a mano. De cada columna se configura el título visible, el orden, el formato, si es visible u ocultable, si es agrupable y qué total lleva al pie.

El formato se sugiere según el tipo SQL de origen: money pasa a moneda, datetime a fecha, decimal(18,2) a número y decimal(18,0) a entero. La consulta en modo diseño atraviesa la misma guarda que al ejecutar: no es una puerta trasera.

> **Detalle de ICG que cuesta caro descubrir**
>
> Los porcentajes no se multiplican por cien. ICG los almacena ya en escala 0-100: un 12,34 % está guardado como 12.34. Multiplicarlo produciría un 1.234 %.

# 6. Capa semántica y tableros

Los tableros no guardan SQL. Guardan referencias a un catálogo de datos, y el motor genera la consulta. Esto tiene dos consecuencias: el usuario final no necesita saber T-SQL, y todo lo que llega a la base de datos proviene de una lista blanca definida por un administrador.

## 6.1 Estructura del catálogo

| Elemento | Qué define |
|---|---|
| Origen | Una consulta base con nombre lógico. De hechos (medibles) o dimensión (describe). |
| Campo | Una columna del origen: rol (dimensión o métrica), tipo, agregación y formato. |
| Relación | Cómo se enlazan dos orígenes, y con qué cardinalidad. |

El catálogo base que se entrega, identificado como ICG-BASE, cubre ventas (cabecera y detalle), artículos, clientes, almacenes, existencias y una dimensión de calendario completa con año, semestre, cuatrimestre, trimestre, mes, semana y año-mes.

## 6.2 El problema del fan-out

Es el error que más silenciosamente corrompe un tablero. Si se unen dos orígenes de hechos —ventas y existencias— por artículo, cada venta se repite por cada fila de stock de ese artículo, y los importes se inflan sin que nada avise.

El motor lo evita agregando cada origen de hechos por separado y uniendo después los resultados ya agregados por las dimensiones comunes. Las dimensiones puras sí se unen directamente, porque la relación es de muchos a uno y no multiplica filas.

> **Verificado con datos reales**
>
> La suma de ventas dio 13.036.181,30 con y sin incorporar existencias al mismo tablero. Sin la corrección, añadir un campo de stock alteraba el total de ventas. Es la prueba de que la preagregación funciona, y conviene repetirla ante cualquier cambio en el motor semántico.

## 6.3 Regla de negocio: FACTORMONEDA

> **No negociable**
>
> En ICG, los importes de un documento están expresados en la moneda del documento. Para obtener el importe en la moneda principal hay que multiplicarlos por el factor de la moneda. Todo origen que exponga importes debe aplicar esa multiplicación en su consulta base.

Omitir este factor no produce ningún error: produce cifras equivocadas, que es peor. Al detectarlo y corregir el catálogo, una medición real pasó de 19,5 millones a 13,0 millones, una desviación del 33 % que nadie habría notado sin contrastar contra el ERP.

Por eso el editor de orígenes muestra un recordatorio permanente junto al campo de la consulta. Al revisar o incorporar un origen con importes, verifíquelo explícitamente.

## 6.4 Propiedad del catálogo

Cada origen lleva marca de propiedad. Los del paquete ICG-BASE son de solo lectura: las actualizaciones de la herramienta pueden corregirlos y mejorarlos. Los creados por el cliente no llevan paquete y no se tocan nunca durante una actualización.

Si un cliente necesita variar un origen base, lo duplica: obtiene una copia propia y editable sin perder las mejoras del original. El catálogo ya evolucionó de la versión 1 a la 3 —corrección de FACTORMONEDA y dimensión de calendario — y los tableros existentes siguieron funcionando, lo que confirma que el mecanismo de versiones es compatible hacia adelante.

## 6.5 Objetos del tablero

- Tablas, gráficos de barras, líneas, áreas, circular y tarjetas de indicador.
- Filtros de lista y de fecha, con períodos relativos que se recalculan en el servidor: "este mes" sigue significando el mes en curso cada vez que se abre el tablero.
- Cada filtro puede ser visible u oculto, y editable o fijo, lo que permite acotar un tablero sin que el usuario pueda ampliarlo.
- Filtrado cruzado en ambos sentidos: al pulsar sobre un gráfico se filtra la tabla, y al seleccionar en la tabla se filtran los gráficos.
- Pestañas para organizar tableros extensos, y disposiciones predefinidas.

# 7. Auditoría, versionado y estadísticas

## 7.1 Bitácora

rep.AuditoriaEvento registra quién hizo qué, cuándo y desde qué dirección IP. Cubre inicios de sesión —incluidos los fallidos —, creación, modificación y borrado de reportes, cada consulta ejecutada, exportaciones, envíos, cambios de programación y todo SQL rechazado.

Guarda el código de usuario de ICG además del identificador interno, de modo que el rastro sobrevive aunque el usuario se elimine de la herramienta. Esta decisión corrigió un defecto real: las estadísticas mostraban cero usuarios porque solo se registraba el identificador interno, nulo para quienes nunca fueron dados de alta explícitamente.

## 7.2 Versionado

Cada cambio guarda una copia completa de la definición del reporte: consulta, parámetros, columnas y opciones. La pantalla de historial muestra la evolución y permite comparar dos versiones, señalando las diferencias en el SQL y los parámetros añadidos, eliminados o modificados.

La tabla no tiene clave foránea contra el reporte, deliberadamente: la historia debe sobrevivir al borrado. El borrado de reportes es lógico por la misma razón.

## 7.3 Estadísticas

Tres vistas alimentan el panel de auditoría: uso por reporte, actividad por usuario y actividad diaria. De ahí salen los reportes más consultados, los usuarios más activos, los reportes más lentos, los errores y —muy útil al depurar un catálogo— los reportes que nadie usa.

# 8. Programación y envíos

El worker no depende de SQL Server Agent ni de Redis, porque ninguno de los dos está garantizado en las instalaciones de destino: SQL Agent no existe en la edición Express, que es la habitual en clientes pequeños.

## 8.1 Funcionamiento

1. Cada minuto encola las programaciones vencidas y recalcula su próxima corrida con la expresión cron.
2. Cada cinco segundos drena la cola, reclamando trabajos con UPDLOCK y READPAST.
3. Cada diez minutos recupera los trabajos que quedaron colgados por un worker caído.
4. Por cada trabajo: ejecutar, exportar, enviar y registrar en el historial.

> **Por qué UPDLOCK con READPAST**
>
> Es lo que permite ejecutar varios workers en paralelo sin coordinación externa. Cada uno bloquea la fila que toma y salta las que ya están bloqueadas, de modo que dos workers nunca procesan el mismo trabajo. Se verificó con cuatro workers simultáneos sobre una cola de doce trabajos: ninguno se procesó dos veces.

## 8.2 Permisos de lo programado

Los envíos programados se ejecutan con los permisos del usuario que creó la programación. El scheduler nunca elude la validación de empresas de ICG: si a ese usuario le retiran el acceso a una empresa, sus envíos programados dejan de poder leerla.

## 8.3 Defectos corregidos

- Las programaciones nuevas se disparaban de inmediato, porque su próxima ejecución nacía nula y se interpretaba como vencida.
- Una expresión cron inválida detenía el bucle completo y con él todas las demás programaciones. Ahora el fallo queda aislado en la programación que lo provoca.

# 9. Exportación y portabilidad

## 9.1 Formatos

PDF, Excel, CSV, HTML, JSON y XML. El PDF permite elegir orientación y tamaño de hoja —A4, A3, carta, oficio y tabloide— tanto como valor por defecto del reporte como puntualmente al exportar.

> **Restricción técnica de pdfmake**
>
> Debe permanecer en la versión 0.2.x. La 0.3 eliminó la interfaz de servidor que se usa aquí (new PdfPrinter(fonts)). Se emplean las fuentes estándar del formato PDF, así que no hay ningún archivo de tipografía que empaquetar ni que descargar.

En HTML el logo se incrusta en base64 dentro del propio archivo, de modo que el documento se ve igual sin conexión y sin recursos externos.

## 9.2 Portabilidad entre clientes

La definición de un reporte se exporta a un archivo JSON e se importa en la instalación de otro cliente. No viajan datos, ni credenciales, ni identificadores internos.

Antes de importar, el archivo se analiza sin guardar nada y se muestra un resumen junto con los problemas detectados. Bloquean la importación: formato desconocido, ausencia de consultas, SQL que no pasa la guarda y —el caso más frecuente al migrar— un procedimiento almacenado que el cliente de origen tenía autorizado y el de destino no. La solución correcta es autorizarlo en la pantalla de procedimientos, nunca relajar la validación.

El SQL importado atraviesa la misma guarda que el escrito a mano, así que un archivo manipulado no puede introducir sentencias destructivas. Cada exportación e importación queda auditada, y la importación crea la primera versión del historial.

# 10. Interfaz

React 18 con Vite. Sin framework de componentes de terceros: la hoja de estilos se apoya en un sistema de tokens propio, lo que evita arrastrar dependencias que caduquen y mantiene el paquete pequeño.

## 10.1 Sistema de diseño

- Tokens centralizados en tokens.css: color, espaciado, radios, tipografía y transiciones.
- Modo claro y oscuro; arranca según la preferencia del sistema operativo y se recuerda.
- Iconografía en SVG propio, 40 iconos. No se usan emojis: cambian de forma según el sistema, no heredan el color del texto y los lectores de pantalla los leen como una palabra suelta.
- Todos los pares de color superan el mínimo de contraste exigido por WCAG AA, en ambos temas.
- Se respeta la preferencia de movimiento reducido del sistema operativo.

> **Tipografía del sistema, a propósito**
>
> No se usan fuentes web. La aplicación debe funcionar en servidores sin salida a internet, y una fuente que no carga degrada la interfaz de forma impredecible. Se emplea la pila tipográfica del sistema, con cifras de ancho fijo para que las columnas numéricas queden alineadas.

## 10.2 Rendimiento

El editor T-SQL (Monaco) y las librerías de tableros (ECharts y la tabla dinámica) se cargan solo cuando se usan. La carga inicial bajó de aproximadamente 1,5 MB a 68 kB comprimidos: quien únicamente consulta reportes no descarga el editor.

> **Monaco debe servirse desde el paquete local**
>
> La librería usa un CDN por defecto. Eso rompería en cualquier servidor sin internet, que es el escenario habitual. La configuración local ya está hecha; conviene no revertirla al actualizar la dependencia.

# 11. Instalación y despliegue

## 11.1 Requisitos

| Componente | Requisito |
|---|---|
| Sistema operativo | Windows Server (o Windows) — también funciona en Linux |
| Base de datos | SQL Server 2016 o superior. Compatible con la edición Express |
| Node.js | Versión 18 o superior |
| Servicio | NSSM para registrar API y worker como servicios de Windows |
| Licencias | Ninguna. Todo el stack es libre de costo |

> **SQL Server 2016 es la línea base**
>
> Todo el T-SQL debe ser compatible con esa versión. No use STRING_AGG, TRIM, ni las cláusulas de JSON introducidas después: hay clientes en 2016 y el script debe instalar sin modificaciones en todos.

## 11.2 Instalación de la base de datos

```
cd db
.\instalar.ps1                          # local, autenticación de Windows
.\instalar.ps1 -Servidor SRV -Usuario sa -Clave "..."   # servidor remoto
```

El script ejecuta los doce archivos SQL en orden, es re-ejecutable — sirve también para actualizar — y verifica la codificación al terminar.

> **Nunca invoque sqlcmd sin -f 65001**
>
> Los archivos .sql están en UTF-8, pero sqlcmd los lee con la página de códigos ANSI del sistema salvo que se le indique lo contrario. Sin ese parámetro, "Código" se almacena como "CÃ³digo" y las etiquetas del catálogo quedan corrompidas en la base. El script instalar.ps1 ya lo aplica; el riesgo aparece al ejecutar un archivo suelto a mano.

## 11.3 Puesta en marcha

```
cd backend
npm install
copy .env.example .env      # y completar los valores reales
npm start                   # API
npm run worker              # scheduler, en otra consola

cd ../frontend
npm install
npm run build               # genera dist/ para servir en producción
```

## 11.4 Configuración

| Variable | Para qué sirve |
|---|---|
| CONFIG_DB_* | Conexión a la base de configuración (lectura y escritura) |
| GENERAL_DB_* | Conexión a GENERAL de ICG (solo lectura) |
| DATA_DB_DEFAULT_* | Credencial por defecto para las bases de empresas |
| APP_CRYPTO_KEY | Llave AES-256 para los secretos guardados en la base. Única por instalación |
| SESSION_SECRET | Firma de la cookie de sesión |
| ADMIN_LOGINS | Usuarios de ICG que siempre son administradores |
| CATALOGO_ADMIN_CODUSUARIOS | Códigos autorizados a administrar el catálogo. Por defecto 0 (SUPERVISOR) |
| QUERY_TIMEOUT_MS | Tiempo límite de consulta. 120.000 ms por defecto |
| WORKER_ID | Identidad del worker en la cola. Debe ser distinta en cada servidor |

> **Contraseñas y dotenv**
>
> Evite los caracteres # $ ' " y los espacios en las contraseñas del .env: dotenv los interpreta y el inicio de sesión falla de forma silenciosa, sin mensaje claro. Es una hora de diagnóstico que se ahorra generando la contraseña solo con letras y números.

# 12. Pruebas

## 12.1 Automáticas

```
cd backend && npm test      # 163 pruebas, sin dependencias externas
```

| Suite | Nº | Cubre |
|---|---|---|
| sqlGuard | 58 | Cargas de inyección, evasión por comentarios, encadenado, EXEC |
| formato | 16 | Formatos de columna y sugerencia según tipo SQL |
| panel | 15 | Objetos del tablero y su configuración |
| export | 14 | Exportación en todos los formatos, con identidad corporativa |
| periodos | 13 | Períodos relativos y su resolución a rangos |
| filtros | 11 | Filtros del tablero, incluido el valor cero como filtro válido |
| engine | 10 | Sustitución de parámetros y enlazado de tipos |
| portabilidad | 9 | Análisis de definiciones importadas |
| relaciones | 9 | Relaciones del catálogo y prevención de fan-out |
| usuarios | 8 | Salvaguardas de roles y del último administrador |

## 12.2 Manuales contra servidor real

No forman parte de npm test porque requieren base de datos y API en marcha:

| Guion | Qué verifica |
|---|---|
| e2e.manual.js | Flujo completo: acceso, empresa, rechazo de SQL peligroso, creación, ejecución, exportación, portabilidad, auditoría y versiones. Limpia lo que crea. |
| scheduler.manual.js | Cuatro workers en paralelo sobre doce trabajos: sin duplicados, reintentos, paso a error y recuperación de huérfanos. |
| semantico.manual.js | Motor semántico contra datos reales, incluida la comprobación de fan-out. |
| catalogoAdmin.manual.js | Alta y edición de orígenes. |
| dashboardUI.manual.js | Construcción de tableros. |
| perfiles.manual.js | Visibilidad por perfil. |

Ninguno imprime contraseñas en pantalla ni en los registros.

# 13. Guía para el responsable entrante

## 13.1 Primeros pasos

1. Clonar el repositorio y crear el .env desde .env.example.
2. Levantar SQL Server con una copia de la base GENERAL de un cliente y al menos una base de empresa.
3. Crear los dos logins de solo lectura y de aplicación (install/windows-service.md).
4. Ejecutar db/instalar.ps1.
5. Arrancar API y frontend, entrar con el usuario SUPERVISOR y recorrer un reporte de ejemplo.
6. Ejecutar npm test para confirmar que el entorno está sano.
7. Ejecutar test/e2e.manual.js contra el servidor real: es la comprobación más completa que existe.

## 13.2 Orden sugerido de lectura del código

| Orden | Archivo | Por qué |
|---|---|---|
| 1 | db/01_schema.sql | El modelo de datos explica el resto del sistema |
| 2 | src/db/connectionManager.js | Cómo se resuelve a qué base se conecta cada consulta |
| 3 | src/services/authService.js | Acceso, permisos y empresas |
| 4 | src/engine/sqlGuard.js | El corazón de la seguridad |
| 5 | src/engine/queryTemplateEngine.js | Cómo se enlazan los parámetros |
| 6 | src/engine/semanticQuery.js | La parte más compleja: el motor de tableros |
| 7 | src/services/schedulerService.js | La cola y el reclamo de trabajos |

## 13.3 Reglas que no deben romperse

| Regla | Consecuencia de romperla |
|---|---|
| **No crear tablas en las bases de ICG** | Una actualización del ERP puede fallar o borrarlas. Rompe el compromiso con el cliente. |
| **No ampliar los permisos del login de datos** | Elimina el único muro que no depende de nuestro código. |
| **No relajar sqlGuard para que "pase" una consulta** | Si una consulta legítima no pasa, reescríbala o conviértala en un SP autorizado. |
| **No concatenar valores en el SQL** | Reabre la inyección por parámetros, que es exactamente lo que el diseño evita. |
| **Aplicar siempre FACTORMONEDA en los importes** | Produce cifras equivocadas sin ningún error visible. |
| **Mantener la compatibilidad con SQL Server 2016** | El script deja de instalar en los clientes que siguen en esa versión. |
| **Ejecutar sqlcmd siempre con -f 65001** | Corrompe los acentos de las etiquetas dentro de la base de datos. |
| **No versionar el archivo .env** | Expone las credenciales de la base de datos del cliente. |

## 13.4 Diagnóstico rápido

| Síntoma | Causa habitual |
|---|---|
| No inicia sesión y no hay error claro | Contraseña del .env con # $ ' " o espacios: dotenv la corta |
| Acentos corruptos en etiquetas | Se ejecutó un .sql sin -f 65001 |
| Un reporte no ejecuta y menciona un SP | El procedimiento no está en la lista blanca |
| Las cifras del tablero no cuadran con ICG | Falta FACTORMONEDA, o hay fan-out entre dos orígenes de hechos |
| Un total cambia al añadir un campo | Fan-out: revise la preagregación en semanticQuery.js |
| Estadísticas con cero usuarios | Ejecuciones sin código de usuario de ICG; revise EjecucionLog |
| Envíos programados que no salen | El worker no está corriendo, o el usuario perdió acceso a la empresa |
| Pantalla en blanco tras una acción | Error de JavaScript; ErrorBoundary lo muestra, revise la consola |

# 14. Pendientes y evolución

## 14.1 Pendiente

| Tema | Detalle | Prioridad |
|---|---|---|
| Ampliar el catálogo | Incorporar compras y contabilidad, y publicarlo como ICG-BASE v4 | Alta |
| Redimensionar objetos | Hoy se ajustan por selectores de ancho y alto; falta arrastrar el borde | Media |
| Mover entre pestañas | Trasladar un objeto de una pestaña a otra dentro del tablero | Media |
| Deslizador de rango | Filtro numérico continuo, al estilo de Power BI | Baja |

## 14.2 Riesgos conocidos

| Riesgo | Mitigación actual |
|---|---|
| Un SP autorizado escribe datos | El login de solo lectura lo impide. Revisar antes de autorizar, con atención a EXECUTE AS |
| Consulta pesada que degrada el ERP | Tiempo límite de 120 s y conexiones con readOnlyIntent, que permiten usar réplicas de lectura |
| Cambio de esquema en ICG | Los orígenes del catálogo se rompen de forma visible; los campos retirados se marcan como obsoletos en lugar de desaparecer |
| Pérdida de la llave de cifrado | Los secretos guardados no se recuperan: hay que volver a introducirlos. Respaldar el .env aparte del repositorio |

## 14.3 Decisiones de diseño y su motivo

| Decisión | Motivo |
|---|---|
| Cola propia en SQL Server | SQL Agent no existe en la edición Express, habitual en clientes pequeños |
| Sin Redis | Sería un servicio más que instalar y vigilar en el servidor del cliente |
| Acceso solo con contraseña | Es el patrón del propio ICG; verificado sin colisiones en la instalación real |
| PATHBD leído en vivo | Evita cadenas de conexión duplicadas que se desfasan al mover una empresa |
| Tableros sin SQL guardado | Todo lo que llega a la base proviene de una lista blanca |
| Borrado lógico de reportes | La auditoría debe sobrevivir al borrado |
| Fuentes y librerías locales | Los servidores de destino suelen no tener salida a internet |

# 15. Referencia rápida

## 15.1 Estructura del repositorio

```
ICG_Reporteria/
 |
 +-- db/                  12 scripts SQL + instalar.ps1
 +-- backend/
 |    +-- src/
 |    |    +-- config/      carga del .env
 |    |    +-- db/          connectionManager.js
 |    |    +-- engine/      sqlGuard - queryTemplateEngine - reportExecutor
 |    |    |                semanticQuery - sqlTypeMapper
 |    |    +-- middleware/  auth.js
 |    |    +-- routes/      10 modulos de API
 |    |    +-- services/    20 servicios de dominio
 |    |    +-- server.js    API
 |    |    +-- worker.js    scheduler
 |    +-- test/             11 suites automaticas + 6 guiones manuales
 +-- frontend/
 |    +-- src/
 |         +-- components/  28 componentes
 |         +-- tokens.css   sistema de diseno
 |         +-- styles.css
 +-- docs/                  este documento y su generador
 +-- install/               servicio de Windows y rotacion de credenciales
 +-- README.md
```

## 15.2 API HTTP

| Prefijo | Función |
|---|---|
| POST /api/login · /api/logout · GET /api/me | Sesión |
| POST /api/sesion/empresa · GET /api/empresas | Contexto de empresa |
| /api/carpetas | Árbol de carpetas |
| /api/reportes | Definición, ejecución, exportación, envío, versiones, portabilidad |
| /api/dashboards | Tableros y consulta semántica |
| /api/catalogo | Orígenes, campos y relaciones |
| /api/programaciones | Programación y historial de envíos |
| /api/usuarios · /api/perfiles | Roles y perfiles |
| /api/procedimientos | Lista blanca de SP |
| /api/branding | Identidad corporativa por empresa |
| /api/auditoria | Bitácora y estadísticas |

## 15.3 Dependencias

| Backend | Uso | Frontend | Uso |
|---|---|---|---|
| express | Servidor HTTP | react 18 | Interfaz |
| mssql | Acceso a SQL Server | @tanstack/react-table | Rejilla de datos |
| pdfmake 0.2.x | PDF (fijar versión) | echarts | Gráficos |
| exceljs | Excel | monaco-editor | Editor T-SQL (local) |
| nodemailer | Correo | react-pivottable | Tabla dinámica |
| cron-parser 5 | Expresiones cron | vite | Compilación |
| express-session | Sesión | | |
| dotenv | Configuración | | |

> **Dos versiones que no deben subirse a la ligera**
>
> pdfmake debe permanecer en 0.2.x: la 0.3 eliminó la interfaz de servidor que se usa. cron-parser está en la 5, cuya API cambió respecto de la 4 (CronExpressionParser.parse). Ambas rompen en tiempo de ejecución, no al compilar.